CSE 213 — Software Engineering

State (Statechart) Diagrams

Lesson notes · Exam approach · TF practice with solutions · Quick notes (Anindya Sir's portion)

How to Read These Notes

EXAM FOCUS	Exam importance, where it appears in the TF collection
MEMORIZE	Rules, tables and formulas to memorize
TRAP	Common mistakes and exam traps
EXAM-READY ANSWER	Model wording you can write in the exam
NOTE	Extra explanation / clarification
Grey box, monospace	Code, worked tables, derivations and hand-drawn sketches
Centered grey box	Formula to remember / final numerical result
Purple heading / ✓ green	Practice questions / their solutions

Contents

Exam Map — What the TF Papers Ask — statechart questions
Lesson 13 — States, Events, Transitions, Actions — sl. 1–11; four event types, Campaign
Lesson 14 — Nested and Concurrent States; Preparing Statecharts — sl. 12–24; graduate-course question
Lesson 15 — Worked Statecharts (garage, order, TF scenarios) — sl. 25–29; insurance, video tape, bank account
Quick Notes — State Diagrams — revision + key diagrams

NOTE Slide numbers refer to the PDF page numbers of the deck named under each lesson title. TF page numbers refer to 'CSE 213 All Past Papers.pdf' (70 pages); the newest paper (22 batch, labelled 2024-25 here) is a separate file, cited as '22 batch p.N'. Old course code: CSE 307. Slide numbers refer to 'Lec 7 - state diagram.pdf' (29 slides, Bennett et al. 'Presentation 15'). Statecharts are written as transition tables and text diagrams: [State] --event [guard] / action--> [State].

Exam Map — What the TF Papers Ask

Session (paper)	Questions in this note file
2016-17 (CSE 307)	Q2(d) Entity classes; statechart of the most important entity class (13) [L15]
2012-13 (CSE 307)	Q7(a) Passing a graduate course — concurrent state diagram (15) [L14]
2011-12 (CSE 307)	Q3(a) Types of events in a statechart (8) [L13] · Q3(b) Insurance customer (defaulter, cancelled, claim) — statechart (14⅔) [L15]
2010-11 (CSE 307)	Q2(b) Lifetime of a video tape in a video store — statechart (12⅔) [L15]

EXAM FOCUS Insurance topic for recent papers: no statechart question since 2016-17, but it was asked in four of the five older papers (2016-17, 2012-13, 2011-12, 2010-11), always as 'draw the statechart of X, with events, conditions and actions'. Learn the transition syntax and the concurrent-region notation; the four event types are a quick 8 marks.

Lesson 13 — States, Events, Transitions, Actions

Slides 1–11 (Lec 7 - state diagram)

1. State (sl. 1–2)

The current state of an object is determined by its attribute values and its links to other objects (a StaffMember is 'probationary' because of its startDate). A state is a condition during the life of an object in which it satisfies some condition, performs some action or waits for some event. A state (statechart) diagram models the event-driven behaviour of one class, subsystem or whole system.

2. Notation

```
  ●  initial pseudostate            ◉  final state
( State )  rounded rectangle         -->  transition
transition label:   event(args) [guard] / action1; action2
internal compartment:   entry / action    exit / action    do / activity    include / submachine
e.g.  GradeRate (sl. 3):
  ● --> (Pending) --when [rateStartDate <= currentDate]--> (Active)
          --when [rateFinishDate <= currentDate]--> (Lapsed) --after [1 year]--> ◉
```

3. The four types of event (sl. 4)

Event	Occurs when	Example
Change event	a condition becomes true (when …)	when [rateStartDate <= currentDate]
Call event	the object receives a call to one of its operations (from another object or itself)	authorized(authorizationCode) — an operation of Campaign
Signal event	the object receives an asynchronous signal	paymentReceived signal from the bank
Elapsed-time event	a set period passes after an event, usually entry to the current state (after …)	after [1 year]

4. Actions and activities (sl. 6–7)

An **action** is short and uninterruptible (on a transition, or entry/exit). An **activity** (do/) runs while the object stays in the state and can be interrupted. Example — DropDownMenu state Menu Visible: entry / displayMenu, exit / hideMenu, do / playSoundClip, internal transition itemSelected() / highlightItem().

5. The Campaign statechart (sl. 5, 10–11)

```
● --> (Commissioned)
(Commissioned) --authorized(authorizationCode) [contract signed] / setCampaignActive()--> (Active)
                                  (on creation: / assignManager(); assignStaff())
(Active) --campaignCompleted() / prepareFinalStatement()--> (Completed)
(Completed) --paymentReceived(payment) [paymentDue - payment > 0]--> (Completed)      recursive
transition
(Completed) --paymentReceived(payment) [paymentDue - payment = 0]--> (Paid)
(Completed) --paymentReceived(payment) [paymentDue - payment < 0] / generateRefund()--> (Paid)
(revised, sl. 11)
(Paid) --archiveCampaign() / unassignStaff(); unassignManager()--> ◉
```

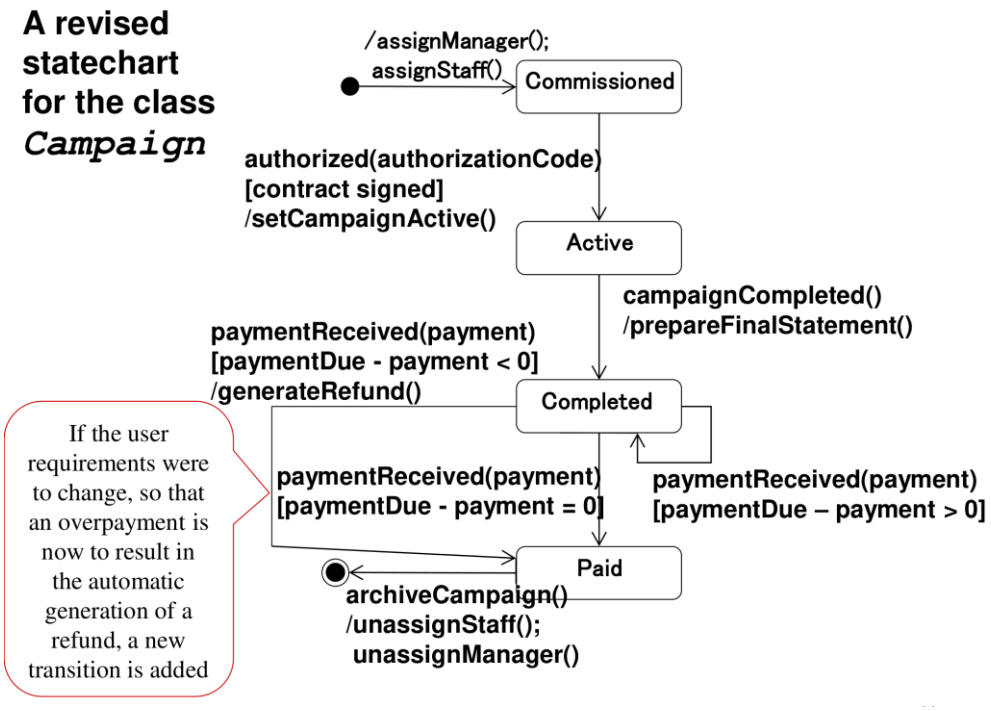


Figure 13.1 — Revised Campaign statechart with refund (sl. 11)

TRAP Guards on the transitions leaving one state for the same event must not overlap and must cover all cases (> 0, = 0, < 0). A guard is a boolean in [ ]; an action follows a slash.

Practice — Lesson 13

(a) Actual — 2011-12 Q3(a).

(a) What are the different types of events in a state chart diagram? (8)

✓ Solution

(a) The four rows of the table in §3, each with its keyword and one example (when for change, operation name for call, asynchronous signal, after for elapsed time). Add one sentence: an event triggers a transition, optionally guarded, which may fire an action.

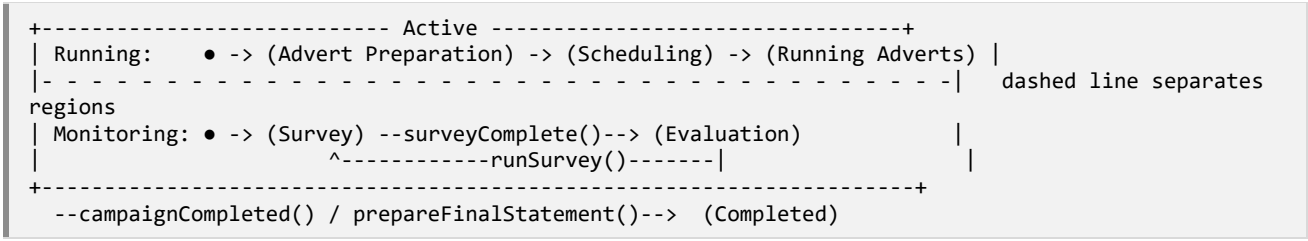
Lesson 14 — Nested and Concurrent States; Preparing Statecharts

Slides 12–24 (Lec 7 - state diagram)

1. Nested substates (sl. 12–13)

A composite state contains its own statechart. Active of Campaign contains Advert Preparation → (advertsApproved() / authorize()) → Scheduling → (confirmSchedule()) → Running Adverts, with extendCampaign() / modifyBudget() back to Advert Preparation. The transition from the inner initial pseudostate carries no event. The detail can be hidden: 'include / Running' plus the decomposition icon (sl. 13).

2. Concurrent substates (sl. 14–17)



- Entering the composite state = entering the initial state of **every** region at once (each region needs an initial state).
- A transition inside one region does not affect the other region.
- A transition out of the composite state leaves all substates, however deeply nested.
- Synchronisation bars (fork / join, sl. 17) show explicitly which substates are entered and that the composite state is left only when both regions have finished.

3. Preparing statecharts (sl. 18–24)

Two approaches: **behavioural** (from interaction diagrams) and **life-cycle** (from the domain description of the object's life). Behavioural approach, 8 steps (sl. 19–20):

1. Examine all interaction diagrams involving each class with heavy messaging.
2. Identify incoming messages that may be events, and the resulting states.
3. Document these events and states on a statechart.
4. Elaborate for additional interactions; add exceptions.
5. Develop nested statecharts.
6. Review against the use cases; check implied constraints.
7. Iterate steps 4–6 until detailed enough.
8. Check consistency with the class diagram, interaction diagrams and other statecharts.

sl. 21–24 apply it to Campaign: the 'Set Campaign Completed' sequence diagram gives campaignCompleted(); the initial, revised and final statecharts add Active as a superstate, the Monitoring region, and campaignCancelled() transitions (/ cancelSchedule(); calculateCosts(); prepareFinalStatement()) from Commissioned and Active.

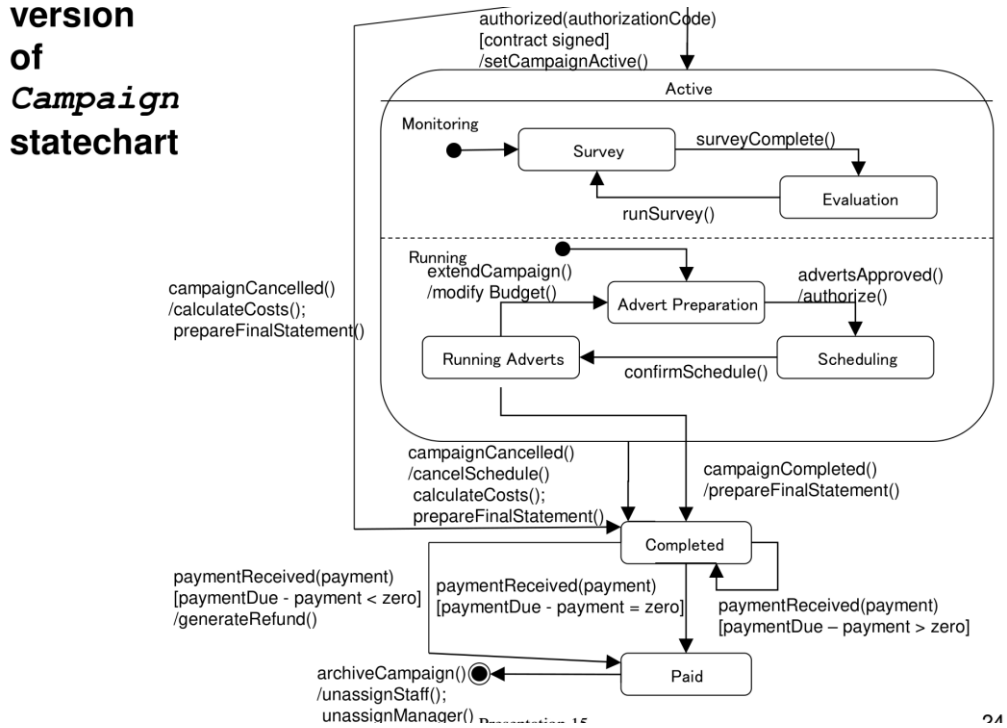


Figure 14.1 — Final Campaign statechart (sl. 24)

Practice — Lesson 14

(a) Actual — 2012-13 Q7(a).

(a) Give a state diagram that describes passing a graduate course as concurrent activities: the student must attend all but two lectures, present a paper she read, and complete a course project due on the last day of term. For the presentation the instructor gives a date, the student prepares and presents on that day. The student can drop the course at any time. Define events, conditions and actions. (15)

✓ Solution

```
● --register()--> [ Taking Course ]  (composite, three concurrent regions)

 Lectures:     ● -> (Attending) --lectureHeld [attended]/ attended++ --> (Attending)
                                --lectureHeld [not attended]/ missed++ --> (Attending)
                   (Attending) --when [missed > 2]--> (Failed Attendance)          (FAIL exit)
                   (Attending) --termEnds [missed <= 2]--> ◉
 - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
 Presentation: ● -> (Waiting for Date) --dateAssigned(d) / record(d)--> (Preparing)
                   (Preparing) --when [today = d] / present()--> (Presented) --> ◉
                   (Preparing) --when [today > d]--> (Missed Presentation)        (FAIL exit)
 - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
 Project:      ● -> (Working on Project) --submit() [today <= lastDayOfTerm]--> (Submitted) --> ◉
                   (Working on Project) --when [today > lastDayOfTerm]--> (Not Submitted) (FAIL exit)

[Taking Course] --all regions reached ◉ (join) / recordPass()--> (Passed) --> ◉
[Taking Course] --any FAIL exit / recordFail()--> (Failed) --> ◉
[Taking Course] --drop() / withdraw()--> (Dropped) --> ◉        (allowed in every substate)
```

Events: lectureHeld, dateAssigned(d), submit(), drop(), termEnds, change events 'when […]'. Conditions: [attended], [missed > 2], [today = d], [today ≤ lastDayOfTerm]. Actions: attended++, record(d), present(), recordPass(), withdraw(). The drop() arrow leaves the composite state, so it applies to all three regions at once.

Lesson 15 — Worked Statecharts (garage, order, TF scenarios)

Slides 25–29 (Lec 7 - state diagram)

1. Garage controller (sl. 25–26)

Events: mode-off / mode-auto / mode-open / mode-closed from the operator; exitgate and entrygate from the gates. The garage counts cars (count) against capacity (max). Gates: locked in closed mode, unlocked in open mode, unspecified when off, and in auto mode locked only when full.

```
(NoCentralControl) --mode-auto / mode-open / mode-closed--> [CentrallyControlled]; any mode-off -->
(NoCentralControl)
[CentrallyControlled] = two concurrent regions
  Gate control: (Manual: Open  entry/unlock) <-> (Manual: Closed entry/lock) ;
                (Automatic: AutoOpen entry/unlock) --entrygate [count >= max]--> (AutoFull entry/lock)
                (AutoFull) --exitgate [count < max]--> (AutoOpen)
  Sign:  (NotFull entry/turn off sign) --entrygate [count = max]--> (Full entry/light sign) --exitgate-->
(NotFull)
  entrygate / count++ ; exitgate / count--   (internal transitions)
```

2. Order processing (sl. 27–29)

```
● -> (Checking  do/check item) --[not all items checked] / get next item--> (Checking)
(Checking) --[all items checked && all available]--> (Dispatching do/initiate delivery) --delivered--
> (Delivered) -> ◉
(Checking) --[all checked && some not in stock]--> (Waiting) --item received [all available]-->
(Dispatching)
Active superstate (sl. 28): cancelled from any of Checking/Waiting/Dispatching --> (Cancelled)
Payment (sl. 29): (Authorizing do/check payment) --[payment OK]--> (Authorized) ; --[payment not OK]-
-> (Rejected)
```

Practice — Lesson 15

(a) Actual — 2011-12 Q3(b). (b) Actual — 2010-11 Q2(b). (c) Actual — 2016-17 Q2(d).

(a) Life insurance: Tk 3,50,000 policy for 5 years; monthly instalment Tk 5000; missing an instalment makes the customer a defaulter; a defaulter pays an extra Tk 500 for each unpaid month to become regular again; 6 consecutive missed instalments cancel the policy; if the customer dies within the period the nominee gets Tk 3,50,000, otherwise the customer gets it at the end. Draw the statechart of a customer. (14⅔)

(b) Lifetime of a video tape: purchased, packaged, entered into the database, put up for rental; rented for 3 days; if not returned the store calls next day, again after 2 more days, and after 2 more days hands it to a collection agency and removes it; damaged on return → removed; missing at annual inventory → removed. Events, conditions, actions. (12⅔)

(c) Bank (normal/foreign users; deposit, withdraw ≤ 1,00,000 per transaction, ≤ 3,00,000 and ≤ 5 withdrawals per day; quarterly interest; 2 % bonus per deposit once balance ≥ 20,00,000; foreign currency conversion): list the entity classes and draw the statechart of the most important one. (13)

✓ Solution

(a) State variable missed = consecutive unpaid months; paidMonths counts paid instalments (60 in 5 years).

```
● --purchasePolicy() / paidMonths := 0; missed := 0--> (Regular)
(Regular)   --payInstalment(5000) / paidMonths++--> (Regular)
(Regular)   --monthEnds [not paid] / missed := 1--> (Defaulter)
(Defaulter) --monthEnds [not paid && missed < 5] / missed++--> (Defaulter)
(Defaulter) --monthEnds [not paid && missed = 5] / missed := 6--> (Cancelled) --> ◉
(Defaulter) --payDues(missed x (5000 + 500)) / paidMonths += missed; missed := 0--> (Regular)
(Regular)   --when [paidMonths = 60] / payCustomer(350000)--> (Matured) --> ◉
(Regular), (Defaulter) --customerDies / payNominee(350000)--> (Claim Settled) --> ◉
```

(b)

```
● --purchase() / package(); addToDatabase()--> (Available for Rent)
(Available) --checkOut(customer) / dueDate := today + 3--> (Rented)
(Rented)    --return() [not damaged]--> (Available)
(Rented)    --return() [damaged] / removeFromDatabase()--> (Removed)
(Rented)    --after [3 days] [not returned] / callCustomer()--> (Overdue 1)
(Overdue 1) --after [2 days] [not returned] / callCustomer()--> (Overdue 2)
(Overdue 2) --after [2 days] [not returned] / sendToCollectionAgency(); removeFromDatabase()--> (Removed)
(Overdue 1), (Overdue 2) --return() [not damaged]--> (Available) ; --return() [damaged] / remove-->
(Removed)
(Available) --annualInventory [missing] / removeFromDatabase()--> (Removed) --> ◉
```

(c) Entity classes: User {abstract} ← NormalUser, ForeignUser; Account; Transaction ← Deposit, Withdrawal, CurrencyConversion; InterestPosting/BonusRule (or attributes of Account). The most important entity is **Account** (all rules change its state).

```
● --createAccount() / balance := 0; withdrawalsToday := 0; withdrawnToday := 0--> [Open]
[Open] has two concurrent regions:
 Balance tier: (Normal) --deposit(a) [balance + a >= 2,000,000]--> (Premium)   entry/ (none)
               (Premium) --deposit(a) / balance += 1.02 a  (2 % bonus)--> (Premium)
               (Premium) --withdraw(a) [balance - a < 2,000,000]--> (Normal)
               (Normal)  --deposit(a) [balance + a < 2,000,000] / balance += a--> (Normal)
 Daily limit:  (Can Withdraw) --withdraw(a) [a <= 100,000 && withdrawnToday + a <= 300,000 && balance >= a]
                    / balance -= a; withdrawnToday += a; withdrawalsToday++--> (Can Withdraw)
               (Can Withdraw) --when [withdrawalsToday = 5 || withdrawnToday = 300,000]--> (Daily Limit
Reached)
               (Daily Limit Reached) --dayEnds / withdrawalsToday := 0; withdrawnToday := 0--> (Can
Withdraw)
[Open]: internal transition  after [3 months] / balance += interest()
[Open] --closeAccount() [balance = 0]--> ◉
```

Exam Approach

1. Pick the object (customer, tape, account) — its life cycle is the diagram.
2. List states as adjectives/conditions (Regular, Defaulter, Rented, Overdue).
3. For each state ask: which events can happen here? where do they lead? write event [guard] / action.
4. Use counters in guards for 'n times' rules; use after [t] for time rules; use concurrent regions for independent rules.
5. Add initial and final states and check every state has a way out.

Quick Notes — State Diagrams

Revision points for Lessons 13–15.

L13 · Basics

- Label: event(args) [guard] / action. entry/, exit/, do/, include/.
- Events: change (when), call (operation), signal (async), elapsed-time (after).
- Campaign: Commissioned → Active → Completed → Paid, guards on paymentDue − payment.

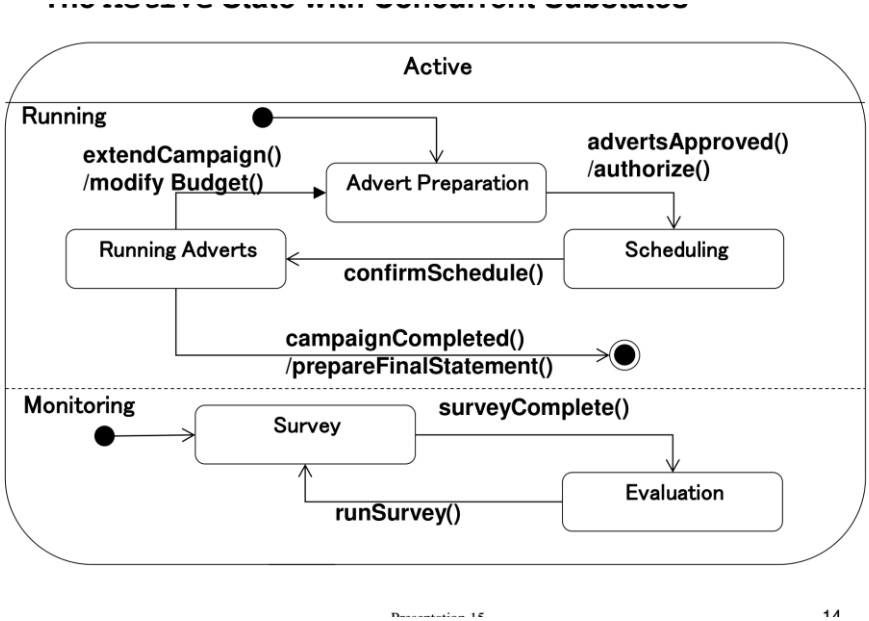
L14 · Nested & concurrent

- Composite state, inner initial state; dashed line between concurrent regions.
- Enter composite = enter all regions; leave composite = leave all; fork/join bars.
- Behavioural approach (8 steps) vs life-cycle approach.

L15 · Worked

- Garage: NoCentralControl / CentrallyControlled (Manual, Automatic) ‖ sign Full/NotFull.
- Insurance: Regular ↔ Defaulter (missed counter) → Cancelled at 6; Matured; Claim Settled.
- Video tape: Available ↔ Rented → Overdue 1 → Overdue 2 → Removed.

Key Diagrams to Reproduce



Concurrent substates of Active (sl. 14)